CHAPTER 1

Introduction to Operating Systems

What an OS is · functions · history · types · boot, interrupts, dual mode · kernel, shell, system calls · OS structures and services · popular operating systems

QUICK TRICK

Learn the fast tricks first (details below):

1. **OS = resource manager + user interface.** Its five big jobs: **process, memory, file, device (I/O)** management and **security**.
2. **Types:** Batch → Multiprogramming → Time-sharing (multitasking) → Real-time → Distributed → Embedded / Mobile.
3. **Dual mode:** user mode (mode bit 1) and kernel mode (mode bit 0). A **system call** uses a **trap** to enter the kernel.
4. **Structures:** Monolithic (UNIX, Linux) · Layered (THE) · Microkernel (QNX, Minix) · Hybrid (Windows NT, macOS).

REMEMBER

How these notes match your exam. The BPSC TRE 4.0 Computer Science (class 11–12 / PGT) paper follows the school and college computer science syllabus. I built this Operating System plan from that syllabus pattern and the standard university OS topics. Please cross-check it with the official BPSC notification, because I could not open the official PDF itself.

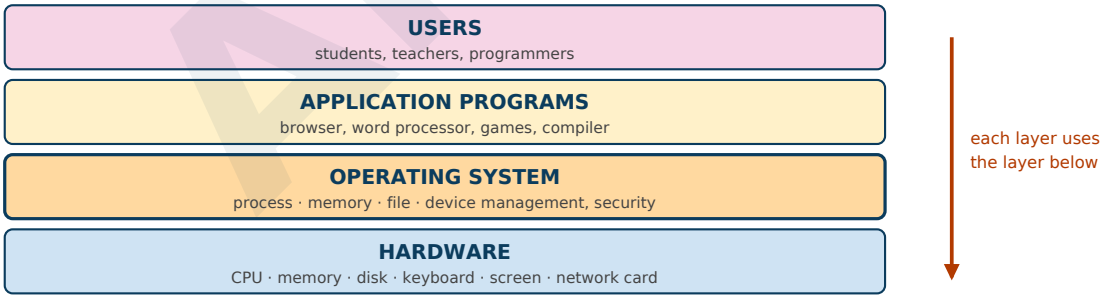
1.1 What is an operating system?

● LEVEL 1 · BEGINNER

The problem. A computer is only metal, chips and wires. Someone must decide which program runs now, where programs are kept in memory, how files are stored, and how the printer is shared. If every program had to do this itself, it would be slow, unsafe and very hard to write. **The idea.** A special program, the **operating system (OS)**, runs all the time, controls the hardware, and gives every other program an easy and safe way to use it.

Definition. An **operating system** is **system software** that **manages the computer hardware and software resources** and provides common services to application programs. It acts as an **interface (middle layer) between the user and the hardware**. The OS is the first program loaded when the computer starts and the last one to stop.

Analogy. A school principal's office. Teachers (programs) need rooms, books and time (resources). The principal (OS) decides who gets what and when, keeps order, and makes sure one class does not disturb another.



The OS sits between the user programs and the hardware

It hides the complex hardware and shares it among programs

Where does the OS fit? Types of software

Kind	Meaning	Examples
System software	Runs and controls the computer; helps other software run	Operating system, device drivers, utility programs (antivirus, disk cleanup), language translators (compiler, interpreter, assembler)
Application software	Does a task the user wants	MS Word, browser, games, Tally, VLC
Firmware	Small permanent program stored in a chip on the hardware	BIOS / UEFI, router firmware
Middleware	Software that helps different applications talk to each other	Database connectors

QUICK TRICK

Two views of an OS: **(1) Resource manager** (it shares CPU, memory, disk and devices fairly and efficiently) and **(2) Extended machine / virtual machine** (it hides ugly hardware details and gives a simple interface such as "save file"). Another term: **control program** (controls execution of programs to prevent errors).

1.2 Goals and functions of an operating system

● **LEVEL 1 · BEGINNER**

Goals: convenience (easy to use), **efficiency** (use the hardware well) and **ability to evolve** (new hardware and features can be added without breaking old software).

Function	What the OS does
Process management	Creates, schedules, runs, suspends and ends **processes** (programs in execution); handles synchronisation and communication; avoids deadlock
Memory management	Keeps track of which part of RAM is used by whom; allocates and frees memory; **virtual memory**, paging, protection
File management	Creates, deletes, reads, writes, names and organises **files and directories** on disk; controls who can access them
Device (I/O) management	Controls devices through **device drivers**; uses buffering, caching and spooling; schedules requests
Secondary storage management	Manages free disk space, disk allocation and disk scheduling
Security and protection	Passwords, user accounts, permissions, encryption; stops one program or user from harming another
User interface	**CLI** (command line) and **GUI** (windows, icons, menus, touch)
Networking	Lets the computer share files, printers and the Internet
Error detection and handling	Detects hardware and software errors (memory fault, printer out of paper) and takes action
Accounting	Keeps records of resource use (CPU time, memory) for billing and statistics

BPSC POINT

The five core managers asked most often: **Process, Memory, File, Device (I/O) and Security**. Each has its own chapter in these notes.

1.3 History and generations of operating systems

● LEVEL 1 · BEGINNER

Generation	Period	Hardware	Software and OS features
First	1940s–1950s	Vacuum tubes, plugboards	**No operating system**; programs in machine language, one user at a time; operators loaded jobs by hand
Second	1950s–1960s	Transistors, mainframes	**Batch systems**: jobs on punched cards were grouped and run one after another by a **resident monitor** (early OS); FORTRAN, COBOL
Third	1960s–1970s	Integrated circuits (IC)	**Multiprogramming** and **time-sharing** (CTSS, MULTICS, OS/360); **UNIX** created at Bell Labs in **1969** (Ken Thompson and Dennis Ritchie)
Fourth	1980s–2000s	Microprocessors, personal computers (PC)	MS-DOS (1981), GUI systems (Mac OS, Windows from 1985), Linux (1991, **Linus Torvalds**), network and distributed OS
Modern	2000s–today	Smartphones, cloud, IoT	**Mobile OS** (iOS 2007, Android 2008), virtualisation and cloud OS, embedded OS for IoT

GNU and Linux. The **GNU project** (Richard Stallman, 1983) built free tools for a Unix-like system but lacked a kernel; **Linux** is the kernel written by Linus Torvalds (first released 1991). Together they form **GNU/Linux**. The kernel is open source, and companies package it as **distributions** (Ubuntu, Debian, Fedora, Red Hat, BOSS).

1.4 Types of operating systems

● LEVEL 1 · BEGINNER

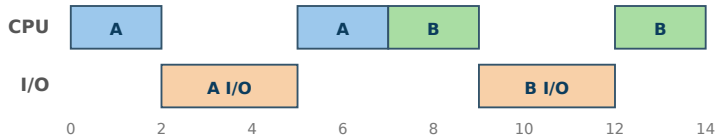
Type	Idea	Good / bad	Examples
Batch	Similar jobs are collected into a **batch** and run one after another with **no user interaction** while running	High throughput; no interaction, long waiting; CPU idle during I/O	Payroll, bank statements, early IBM systems
Multiprogramming	Several programs are kept in memory; when one waits for I/O the CPU runs another	High CPU use; needs memory management and CPU scheduling	Most modern OS; mainframes
Time-sharing (multitasking)	CPU is switched among programs so quickly (every few ms, the **time quantum**) that each user feels the computer is theirs; **interactive**	Good response time; needs good scheduling and protection	UNIX, Linux, Windows, macOS
Multiprocessing (parallel)	Two or more CPUs share memory and work together. **SMP** (symmetric): all CPUs equal; **AMP** (asymmetric): one master, others slaves	More throughput and reliability; costly, complex	Servers, multicore PCs
Real-time (RTOS)	Must respond within a **fixed deadline**. **Hard**: missing a deadline is a failure. **Soft**: occasional miss is tolerable	Predictable; limited features	Hard: pacemaker, airbag, missile control. Soft: video streaming, online games. VxWorks, FreeRTOS, QNX
Distributed	Many computers connected by a network look like **one system** to the user; resources and load are shared	Reliable, scalable; complex	Large clusters, cloud systems
Network OS	Runs on a server and lets many computers share files, printers and users; users know about the separate machines	Central control	Windows Server, Novell NetWare, Linux server
Embedded	Built into a device to do a fixed job; small and efficient	Fast, tiny; not general-purpose	Washing machine, ATM, smart TV, router
Mobile	For phones and tablets: touch, battery saving, sensors	—	Android, iOS

AMJESH

Term	Meaning
Single-user / multi-user	One person at a time (MS-DOS) / many users with separate accounts at the same time (UNIX, Linux server)
Single-tasking / multitasking	One program at a time / several programs appear to run together
Multiprogramming vs multitasking	Multiprogramming aims at **CPU utilisation** (switch when a job waits for I/O). Multitasking (time-sharing) aims at **response time** (switch on a timer, even if the job is not waiting)
Degree of multiprogramming	The number of processes in memory at one time
Throughput	Number of jobs completed per unit time

Why multiprogramming helps: CPU utilisation

WITHOUT multiprogramming (one job at a time): total 14 units, CPU busy 8/14 = 57%



WITH multiprogramming (CPU switches to B while A waits): total 9 units, CPU busy 8/9 = 89%



Formula. If each process spends a fraction **p** of its time waiting for I/O, and there are **n** processes in memory, then **CPU utilisation = 1 − p^n** (the CPU is idle only when all n are waiting).

Example. p = 0.8 (80% I/O wait). n = 1: 1 − 0.8 = **20%**. n = 2: 1 − 0.64 = **36%**. n = 4: 1 − 0.8^4 = 1 − 0.4096 = **59%**. n = 6: 1 − 0.262 = **74%**. More programs in memory give higher CPU use, but with diminishing returns.

COMMON MISTAKE

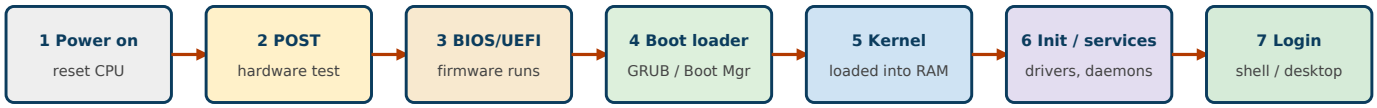
- Mixing **multiprogramming** (CPU switch on I/O wait; many programs in memory), **multitasking** (CPU switch on a timer) and **multiprocessing** (many CPUs).
- Saying real-time means "very fast". It means **meets a deadline predictably**, not necessarily high speed.

1.5 Computer system basics the OS depends on

● LEVEL 2 · EXAM

Booting

Booting is the process of starting a computer and loading the OS into **main memory (RAM)**. A small **bootstrap program** stored in firmware starts it, because RAM is empty and the OS lives on disk. **Cold boot** = start from power off; **warm boot** = restart without cutting power.



Booting = starting the computer by loading the operating system into main memory

Step	What happens
POST	**Power-On Self-Test**: checks CPU, memory, keyboard and other hardware
BIOS / UEFI	Firmware stored in a chip; finds the boot device (disk, USB). **UEFI** is the modern replacement of the older **BIOS** and supports big disks and secure boot
Boot loader	A small program (for example **GRUB** on Linux, Windows Boot Manager) found at the start of the disk (MBR or EFI partition); it loads the kernel
Kernel starts	The kernel is loaded into RAM, sets up memory, devices and the first process
Init and login	System services and drivers start, then the login screen or shell appears

Interrupts

An **interrupt** is a signal that tells the CPU to stop what it is doing, handle an urgent event, and then continue. This avoids **polling** (the CPU repeatedly asking "are you ready?"). The CPU saves its state, jumps to the right **interrupt service routine (ISR)** found through the **interrupt vector** table, and then returns.

Kind	Cause	Example
Hardware interrupt	From a device, asynchronous	Keyboard key press, disk finished reading, **timer** tick
Software interrupt (trap)	Caused by a program on purpose	**System call**
Exception	Caused by an error in a program	Divide by zero, invalid memory access (page fault)

Dual-mode operation and protection

To stop a faulty or bad program from damaging the system, the CPU has **two modes**, shown by a **mode bit** in the hardware:

Mode	Mode bit	Who runs	Power
Kernel mode (supervisor, system, privileged)	0	The operating system	Can run **all** instructions and touch all hardware and memory
User mode	1	Application programs	Can run only safe instructions; cannot directly use I/O or other programs' memory

Privileged instructions can run only in kernel mode: I/O instructions, setting the timer, halting the CPU, changing memory-protection registers, disabling interrupts. If a user program tries one, the hardware raises a **trap** and the OS stops the program. A **timer** interrupt gives the OS control back after a fixed time, so a program cannot keep the CPU forever (for example in an infinite loop).

> **QUICK TRICK**
> **User → kernel** happens by a **trap / system call / interrupt**. **Kernel → user** happens when the OS returns control. Remember **kernel = 0, user = 1**.

Memory hierarchy (why the OS needs memory management)

Level	Speed	Size	Cost per byte	Volatile?
Registers	Fastest	Bytes	Highest	Yes
Cache	Very fast	KB–MB	High	Yes
Main memory (RAM)	Fast	GB	Medium	Yes
Secondary storage (SSD, HDD)	Slow	GB–TB	Low	No

AMJESH

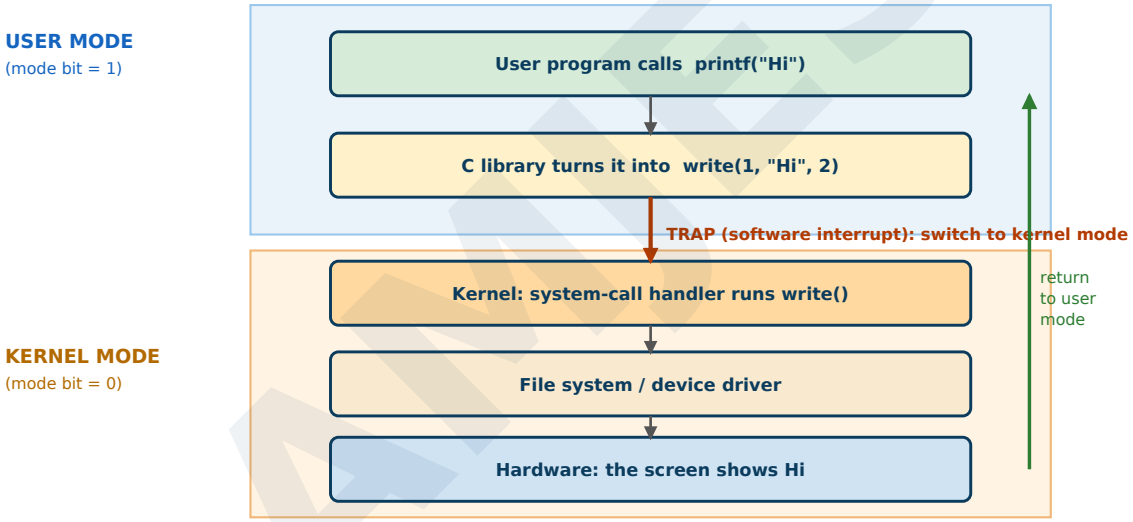
1.6 Kernel, shell, user interface and system calls

● LEVEL 2 · EXAM

Part	Meaning
Kernel	The **core** of the OS. It always stays in memory and runs in kernel mode: handles processes, memory, devices and system calls
Shell	The **command interpreter**: the outer layer that takes the user's commands, passes them to the kernel and shows the result (bash, zsh, Windows Command Prompt, PowerShell). A GUI desktop is also a shell
Device driver	Program that tells the OS how to talk to one particular device (printer, graphics card); usually part of the kernel
Utilities	Helper programs that come with the OS (file manager, task manager, disk cleanup)

User interface	Meaning	Example
CLI (Command Line Interface)	User types text commands. Powerful, light on resources, can be automated by scripts	Linux terminal, Windows cmd
GUI (Graphical User Interface)	Windows, icons, menus and a pointer (WIMP); easy for beginners	Windows desktop, GNOME, macOS
Touch / voice interface	Gestures and speech	Android, iOS, voice assistants

System call. A **system call** is the way a **user program asks the kernel for a service** that needs privilege (open a file, create a process, send data). Programmers mostly use an **API** (Application Programming Interface: library functions such as printf, or the Windows API / POSIX), and the library makes the real system call. **POSIX** is a standard set of UNIX-like system calls.



Category	Purpose	Examples (UNIX / Linux)
Process control	Create, end, load, wait for processes	**fork()**, **exec()**, **exit()**, **wait()**, kill()
File management	Create, open, read, write, close, delete files	**open()**, **read()**, **write()**, **close()**, unlink()
Device management	Request and release devices, read and write them	ioctl(), read(), write()
Information maintenance	Get or set system data	getpid(), time(), alarm(), sleep()
Communication	Send messages between processes or computers	pipe(), socket(), shmget(), msgsnd()
Protection	Control access permissions	chmod(), chown(), umask()

API	System call
Set of library functions for programmers (printf, fopen, Windows API)	The actual entry point into the kernel (write, open)
Runs in user mode; may or may not call the kernel	Always causes a **trap** and a switch to kernel mode

COMMON MISTAKE
A normal function call stays in user mode. A **system call** changes the mode from user to kernel (through a trap) and back. Also, **kernel** and **shell** are not the same: the shell talks to the user, the kernel talks to the hardware.

1.7 Structures of operating systems

● LEVEL 2 · EXAM

The **structure** tells how the OS parts are arranged and how they connect.

Structure	Idea	Good	Bad	Examples
Simple (no structure)	No clear parts; programs can reach hardware directly	Small and fast	No protection; one error crashes everything	MS-DOS
Monolithic	The **whole OS runs as one big program in kernel mode**: scheduler, memory, file system, drivers all together	Fast (direct calls)	Hard to maintain; a bug in a driver can crash the system	Traditional UNIX, Linux (with loadable modules)
Layered	OS divided into layers 0 (hardware) to N (user interface); a layer uses only the layer below	Easy to debug and verify	Hard to decide layers; slower (many layers to pass)	THE system (Dijkstra)
Microkernel	Only the **minimum** (scheduling, memory basics, inter-process communication) in the kernel; file system, drivers and others run as **user-mode servers** that exchange **messages**	Reliable, secure, easy to extend and port	Slower because of message passing	Mach, QNX, Minix
Modular	Core kernel plus **loadable modules** added or removed at run time	Flexible	—	Linux modules, Solaris
Hybrid	Mix of monolithic speed and microkernel ideas	Good compromise	Complex	Windows NT, macOS / iOS (XNU)
Virtual machine	A **hypervisor** makes one computer behave like several separate machines, each running its own OS	Isolation, testing, cloud	Some overhead	VMware, VirtualBox, KVM

QUICK TRICK
Monolithic = fast but risky. Microkernel = safe but slower. Linux is monolithic (with modules); Windows and macOS are hybrid.

1.8 Services provided by an operating system

● LEVEL 2 · EXAM

Services for the user (convenience)	Services for system efficiency
User interface (CLI, GUI) **Program execution**: load, run and end programs **I/O operations**: programs use devices through the OS **File-system manipulation**: create, delete, read, write, search **Communication**: between processes on one or more computers **Error detection**: memory, I/O, program errors	**Resource allocation**: CPU, memory, files, devices to many users and programs **Accounting** (logging): who used how much **Protection and security**: control access, protect against outsiders

1.9 Popular operating systems

● LEVEL 1 · BEGINNER

OS	Maker / origin	Features and notes
Windows	Microsoft; first version 1985 (Windows 1.0)	Proprietary, GUI, desktop and server versions; file system NTFS; hybrid kernel (NT)
Linux	Linus Torvalds (1991); open source	Free, secure, used on servers, supercomputers and Android; distributions Ubuntu, Debian, Fedora, Red Hat; file system ext4
UNIX	AT&T Bell Labs (1969)	Multi-user, multitasking; ancestor of Linux, macOS; written in C
macOS	Apple	Unix-based, GUI; file system APFS; for Mac computers

OS	Maker / origin	Features and notes
Android	Google (first phone 2008)	Linux-based, open source core (AOSP), for phones and tablets
iOS	Apple (2007)	Proprietary, for iPhone and iPad
Chrome OS	Google	Light, browser-centred OS for Chromebooks
BOSS	C-DAC, India	Indian Debian-based Linux distribution (Bharat Operating System Solutions)
RTOS	Various	VxWorks, FreeRTOS, QNX for real-time embedded systems

Point	Proprietary (closed source)	Open source
Source code	Hidden	Available to read, change and share
Cost	Usually paid licence	Usually free
Support	From the company	Community and companies
Examples	Windows, macOS, iOS	Linux, Android (core), FreeBSD

1.10 Common mistakes

● **LEVEL 2 · EXAM**

COMMON MISTAKE

- Saying the OS is an application program. It is **system software**.
- Mixing multiprogramming, multitasking and multiprocessing.
- Saying kernel mode bit is 1. **Kernel = 0, user = 1**.
- Saying the shell is the core of the OS. The **kernel** is the core; the shell is the interface.
- Believing Windows is a monolithic or microkernel system. It is a **hybrid**.
- Treating Linux as a complete OS. Linux is the **kernel**; a distribution adds tools and desktop.
- Saying a hard real-time system is just fast. It must meet **strict deadlines**.

Quick-Solve Tricks — Summary

Situation	Trick
OS definition	System software; resource manager; interface between user and hardware
Core functions	Process, memory, file, device (I/O), security (plus user interface, networking, accounting)
History	No OS → Batch → Multiprogramming / time-sharing (UNIX 1969) → PC and GUI (MS-DOS 1981, Linux 1991) → Mobile (iOS 2007, Android 2008)
Types	Batch, multiprogramming, time-sharing, multiprocessing, real-time (hard / soft), distributed, network, embedded, mobile
CPU utilisation	$1 - p^n$ (p = I/O wait fraction, n = number of processes)
Boot order	Power on → POST → BIOS/UEFI → boot loader → kernel → init → login
Interrupt kinds	Hardware (device, timer), software (trap / system call), exception (error)
Dual mode	Kernel = 0 (all instructions) · User = 1 (restricted); trap switches to kernel
System call types	Process control, file, device, information, communication, protection
Structures	Monolithic · Layered · Microkernel · Modular · Hybrid · Virtual machine
Kernel vs shell	Kernel = core, talks to hardware · Shell = interface, talks to user

AMJESH

BPSC QUICK CHECK — Chapter 1

OS → resource manager, system software	UNIX → 1969, Bell Labs
Linux → kernel by Linus Torvalds (1991)	Windows NT → hybrid kernel
Hard real-time → pacemaker, airbag	Time-sharing → response time, quantum
Kernel mode bit → 0	System call → trap to kernel
Microkernel → QNX, Minix	Monolithic → UNIX, Linux

Traps: multiprogramming vs multitasking vs multiprocessing · kernel vs shell · mode bit values · API vs system call · Linux (kernel) vs distribution.

Practice questions

● LEVEL 1 · BEGINNER

- **Q1.** Define an operating system.
- **Q2.** Name four functions of an operating system.
- **Q3.** Who developed the Linux kernel, and in which year was it first released?
- **Q4.** In which mode can privileged instructions run?
- **Q5.** Name two operating systems used on mobile phones.
- **Q6.** What is a shell?

● LEVEL 2 · EXAM

- **Q7.** Differentiate between multiprogramming and time-sharing (multitasking).
- **Q8.** Differentiate between hard and soft real-time systems. Give one example of each.
- **Q9.** Explain dual-mode operation. Why is it needed?
- **Q10.** What is a system call? Give two examples.
- **Q11.** Give three differences between a monolithic kernel and a microkernel.
- **Q12.** List the steps of booting a computer.

● LEVEL 3 · BPSC / PGT

- **Q13.** Processes spend 80% of their time waiting for I/O. Find the CPU utilisation with 1, 2, 4 and 6 processes in memory. How many processes are needed for at least 90% utilisation?
- **Q14.** Two jobs A and B each need 2 units of CPU, then 3 units of I/O, then 2 units of CPU. Find the total time and CPU utilisation (a) when they run one after the other, (b) with multiprogramming (B uses the CPU while A does I/O).
- **Q15.** Trace what happens when a program calls printf("Hi") on Linux.
- **Q16.** Name the structure of each: (a) QNX, (b) traditional UNIX, (c) Windows NT, (d) THE system, (e) MS-DOS.
- **Q17.** Why is a timer interrupt needed in an operating system?
- **Q18.** Identify the type of system: (a) car airbag controller, (b) online video streaming, (c) bank's night-time payroll run, (d) a cluster of many computers acting as one, (e) smartphone.
- **Q19.** Differentiate between an API and a system call.
- **Q20.** Why is an OS called a resource manager and also an extended machine?

AMJESH

Answers

Q	Solution
1	System software that manages hardware and software resources and provides services to programs; it acts as an interface between the user and the hardware.
2	Process management, memory management, file management, device (I/O) management (also security, user interface, networking).
3	Linus Torvalds; 1991.
4	Kernel mode (supervisor mode, mode bit 0).
5	Android and iOS.
6	The command interpreter, the outer layer of the OS that takes the user's commands and passes them to the kernel (for example bash, Command Prompt).
7	Multiprogramming: many programs in memory, CPU switches when a program waits for I/O; aim is high CPU utilisation. Time-sharing: CPU switches on a timer (time quantum) among interactive users; aim is good response time.
8	Hard real-time: a missed deadline is a failure (pacemaker, airbag, missile control). Soft real-time: an occasional missed deadline is acceptable (video streaming, online games).
9	The CPU runs in user mode (bit 1) for applications and kernel mode (bit 0) for the OS. Privileged instructions run only in kernel mode. This protects the OS and other programs from faulty or malicious programs.
10	A request from a user program to the kernel for a privileged service. Examples: open(), read(), write(), fork(), exit().
11	Monolithic: whole OS in kernel mode, fast, a driver bug can crash all. Microkernel: only minimum in kernel, services in user space, communicate by messages, more reliable but slower.
12	Power on → POST → BIOS/UEFI finds boot device → boot loader loads the kernel into RAM → kernel starts, initialises devices → system services start → login or desktop.
13	Utilisation = $1 - 0.8^n$. n = 1: **20%**; n = 2: **36%**; n = 4: **59%**; n = 6: **73.8%**. For ≥ 90% need $0.8^n \le 0.1$: $n \ge \ln 0.1 \div \ln 0.8 = 10.3$, so **11 processes** (n = 10 gives 89.3%, n = 11 gives 91.4%).
14	(a) A: CPU 0–2, I/O 2–5, CPU 5–7; then B: CPU 7–9, I/O 9–12, CPU 12–14. Total **14 units**; CPU busy 8 → 8 ÷ 14 = **57%**. (b) A CPU 0–2; B CPU 2–4 while A does I/O (2–5); B I/O 4–7; A CPU 5–7; B CPU 7–9. Total **9 units**; CPU busy 8 ÷ 9 = **89%**.
15	The program calls printf in the C library, which formats the text and calls the write() system call. A trap (software interrupt) switches the CPU to kernel mode; the kernel's handler passes the request to the file system and device driver, the screen shows the text, and control returns to user mode with the result.
16	(a) Microkernel, (b) monolithic, (c) hybrid, (d) layered, (e) simple (no structure).
17	It gives control back to the OS after a fixed time, so a program cannot hold the CPU forever (for example in an infinite loop). It also makes time-sharing and scheduling possible.
18	(a) Hard real-time, (b) soft real-time, (c) batch, (d) distributed, (e) mobile OS.
19	An API is a set of library functions used by programmers (printf, fopen); it may or may not enter the kernel. A system call is the actual entry into the kernel, causing a trap and a switch to kernel mode.
20	It is a resource manager because it allocates and shares the CPU, memory, files and devices among programs fairly and efficiently. It is an extended machine because it hides complex hardware details and offers a simpler, easier interface.

REMEMBER

Next: Chapter 2 — Process Management (process concept and states, PCB, context switch, process creation with fork, threads and multithreading models, inter-process communication).

Planned roadmap: Ch 3 CPU Scheduling (FCFS, SJF, Priority, Round Robin with Gantt chart numericals) · Ch 4 Process Synchronisation (critical section, semaphores, classical problems) · Ch 5 Deadlocks (Banker's algorithm) · Ch 6 Memory Management (paging, segmentation) · Ch 7 Virtual Memory (page replacement) · Ch 8 File Systems · Ch 9 I/O and Disk Management (disk scheduling, RAID) · Ch 10 Protection, Security, Linux and Windows basics.

AMJESH